Logistic Regression

Logistic Regression is a popular supervised learning algorithm used in machine learning for **binary classification tasks**. It is mainly employed when the target variable is categorical with two possible classes or labels, typically represented as 0 and 1 or "Yes" and "No".

For example

1) Fraud or Not
2) Email spam or not Spam
3) Infected with Disease or not Disease.

There are two types of approaches to compute logistic regression Graphical and Probabilistic approach.

Binary Class	Multi Class
from sklearn.linear_model import LogisticRegression model = LogisticRegression(solver='liblinear', max_iter=1000)	from sklearn.linear_model import LogisticRegression model = LogisticRegression(**multi_class='ovr'**, solver='liblinear', max_iter=1000)

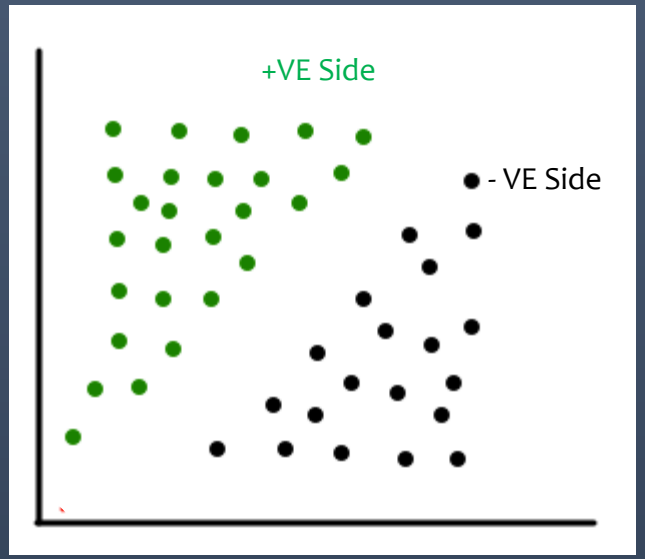
Steps of Logistic Regression

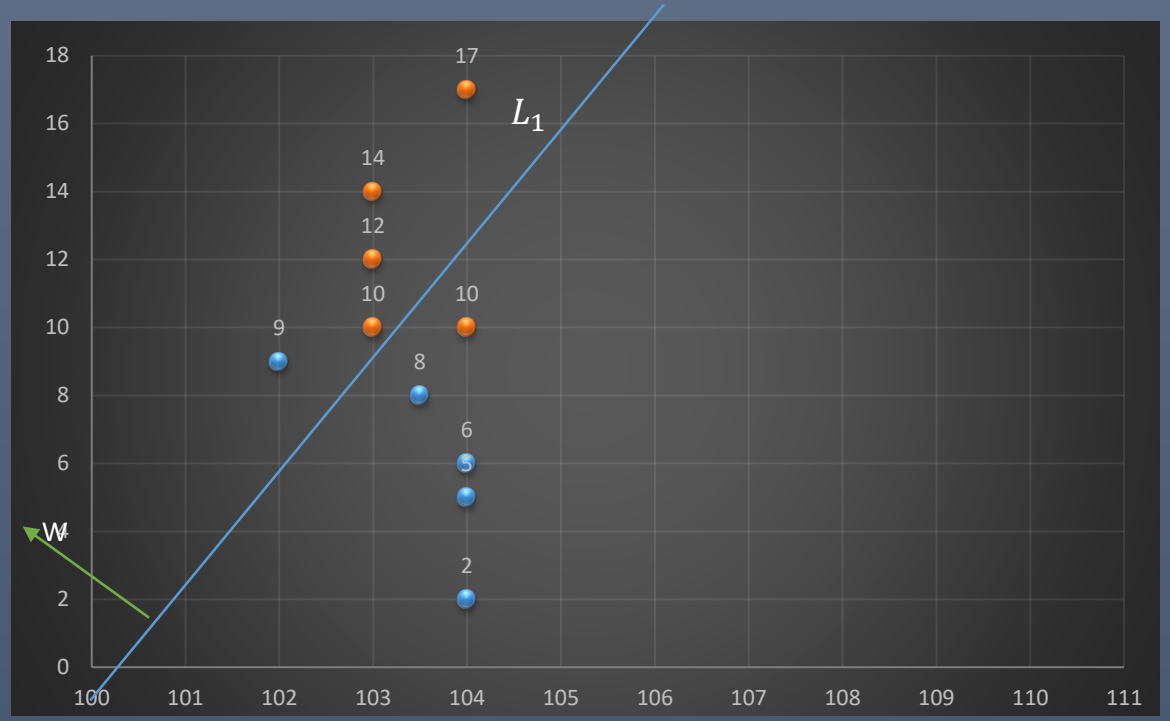
- A best-fit line is established to distinguish between data points.
- Creation of this best-fit line, the next step involves determining whether data points lie on the positive or negative side of the line.
- Then we decide if the point is correctly classified or not.
- Compute the cost Using Sigmoid function.
- Line with highest value to be considered the final line.

In this context, we consider the upper portion of the line as the positive side, and the lower portion of the line as the negative side.

Assume that below is the line best-fit line created using

$y = mx + c$





$$\left\{ \begin{array}{c} Actual = Y_i \\ predicted = d_i \end{array} \right\}$$

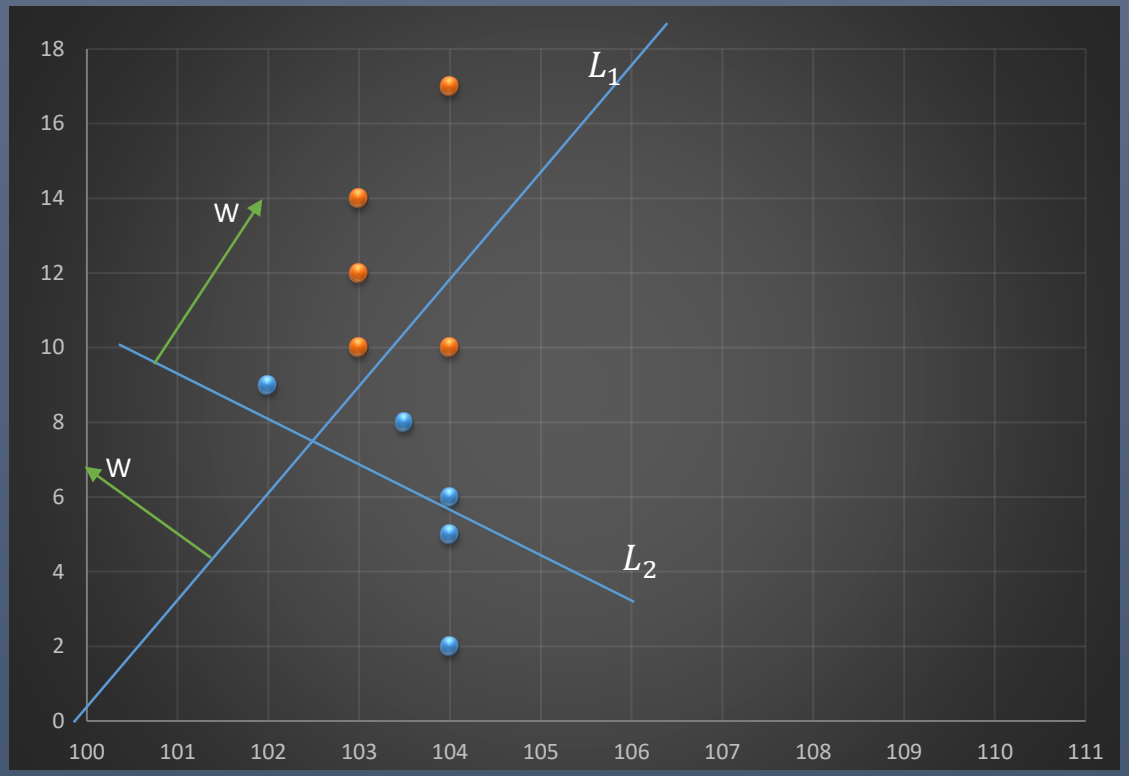
$$(Correctly\ Classified\ Points)$$

$$\left\{ \begin{array}{l} Y_i(+ve) * d_i(+ve) \\ Y_i(-ve) * d_i(-ve) \end{array} \right\} = +ve\ output$$

$$Incorreclty\ Classfied\ Points$$

$$\left\{ \begin{array}{l} Y_i(+ve)\ d_i(-ve) \\ Y_i(-ve)\ d_i(+ve) \end{array} \right\} = -ve\ Output$$

Reading 01	Reading 02	+VE Cancer Patient (1 and 0)	Actual+VE Cancer Patient (+1 and -1)	Predicted as per L1	+VE OR -VE Result	Correct Incorrect
104	17	1	+1	+1	1	Correct
103	14	1	+1	+1	1	Correct
103	12	1	+1	+1	1	Correct
103	10	1	+1	+1	1	Correct
104	10	1	+1	-1	-1	Incorrect
102	9	0	-1	+1	-1	Incorrect
103.5	8	0	-1	-1	1	Correct
104	6	0	-1	-1	1	Correct
104	5	0	-1	-1	1	Correct
104	2	0	-1	-1	1	Correct



Steps of Identifying the best-fit line

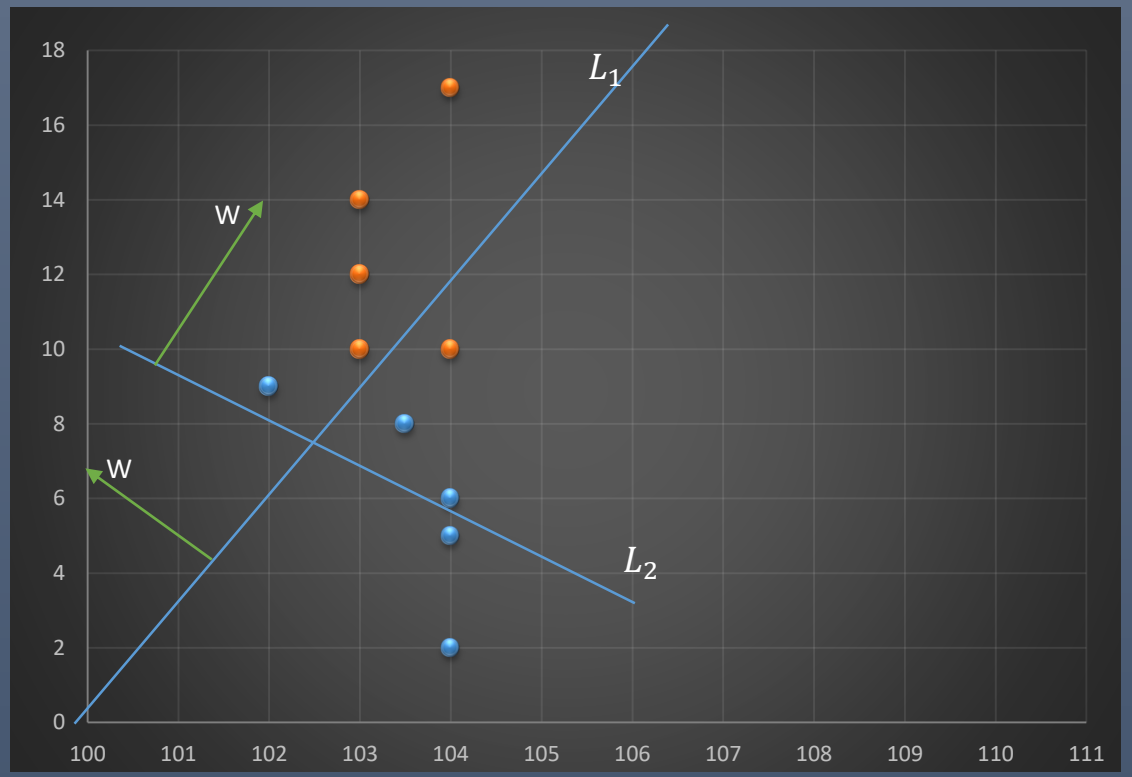
- Create the two or more line as shown in the figure.
- we will multiply the actual and predicted values for each line and calculate the sum separately for each line.
- Then compare the result
- This calculation will yield a numerical value.
- Next, we will compare the calculated values for each line and select the one with the highest output.
- Additionally, we will address any issues related to distance during this evaluation.
- We will use the same equation for the L1 and L2 and see which is the best-fit line by using the below equation.

The same can be identified by using the below equation.

$$\sum_{i=1}^{n} Y_i d_i$$

$$Y_i = Actual\ Points$$

$$d_i = Predected\ points$$



Calculation

$$\sum_{i=1}^{n} Y_i d_i$$

$$= Y_1 d_1 + Y_2 d_2 + Y_3 d_3 + Y_4 d_4 + Y_5 d_5 + Y_6 d_6 + Y_7 d_7 + Y_8 d_8 + Y_9 d_9 + Y_{10} d_{10}$$

For L1

$$= (+1 * +1) + (+1 * +1) + (+1 * +1) + (+1 * +1) + (+1 * -1) + (-1 * +1) + (-1 * -1) + (-1 * -1) + (-1 * -1) + (-1 * -1)$$

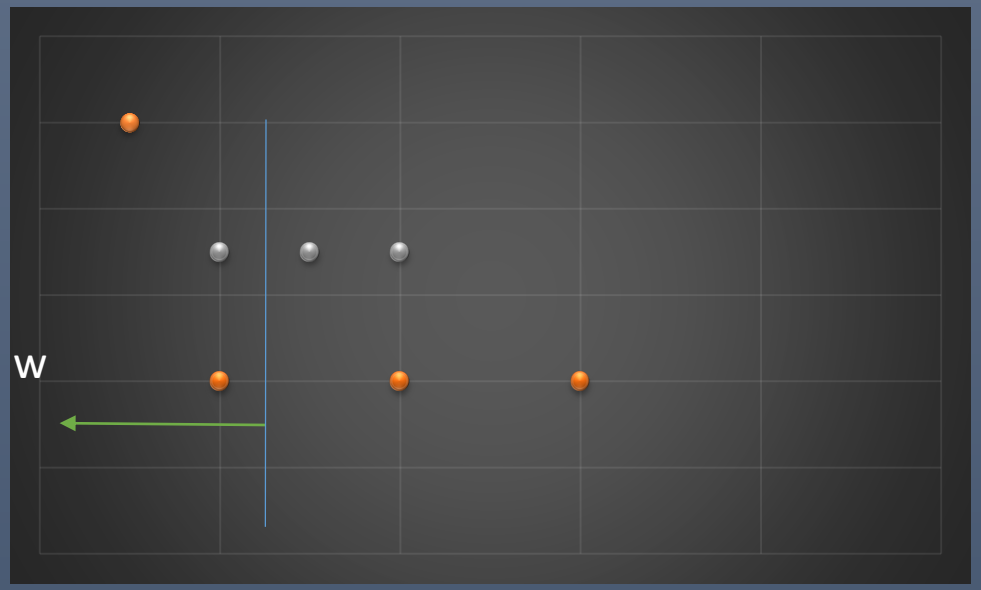
$$= 8 + \boldsymbol{ve\ Points}$$

For L2

$$= (1 * 1) + (1 * 1) + (1 * 1) + (1 * 1) + (1 * 1) + (-1 * 1) + (-1 * 1) + (-1 * 1) + (-1 * 1) + (-1 * -1)$$

$$= 4 + \boldsymbol{ve\ Points}$$

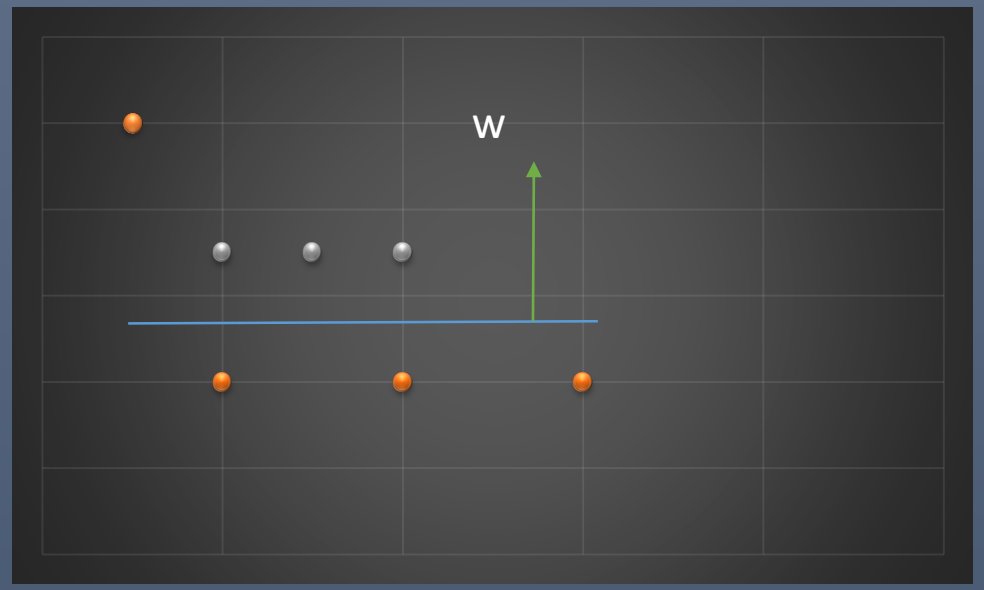
Hence model will select L1 as the final Line,



$$\sum_{i=1}^{n} Y_I d_i$$

Original x L1	Final	
1 * -1 = -1	-4	2
1 * -1 = -1		
1 * -2 = -2		
-1 * 1 = 1	6	
-1 * -5 = 2		
-1 * -6 = 4		
-1 * 1 = -1		

$$\sum_{i=1}^{n} Y_I d_i = 5$$



$$\sum_{i=1}^{n} Y_I d_i$$

Original x L1	Final	
1 * 1 = 1	3	-20
1 * 1 = 1		
1 * 1 = 1		
-1 * 1 = -1	-23	
-1 * 1 = -1		
-1 * 1 = -1		
-1 * 20 = -20		

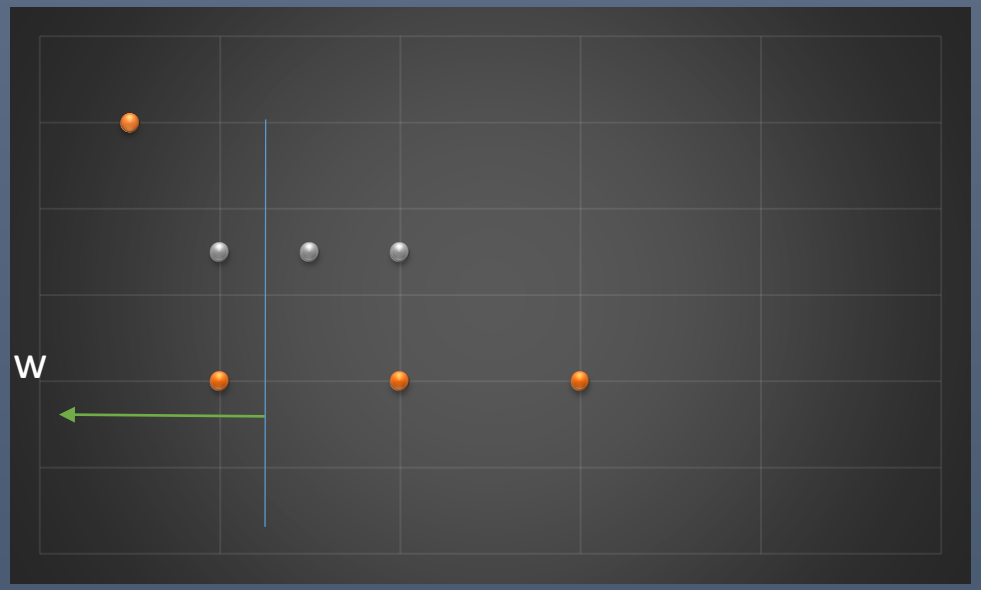
$$\sum_{i=1}^{n} Y_I d_i = -20$$

Certainly, you may have noticed that although the L2 line divides the data points accurately, still it won't be chosen as the final line or the most accurate line.

This is because the total outcome of L2 is (-20) is inferior to the L1.

Hence to overcome such a scenario, we will introduce adjustments to the input approach.

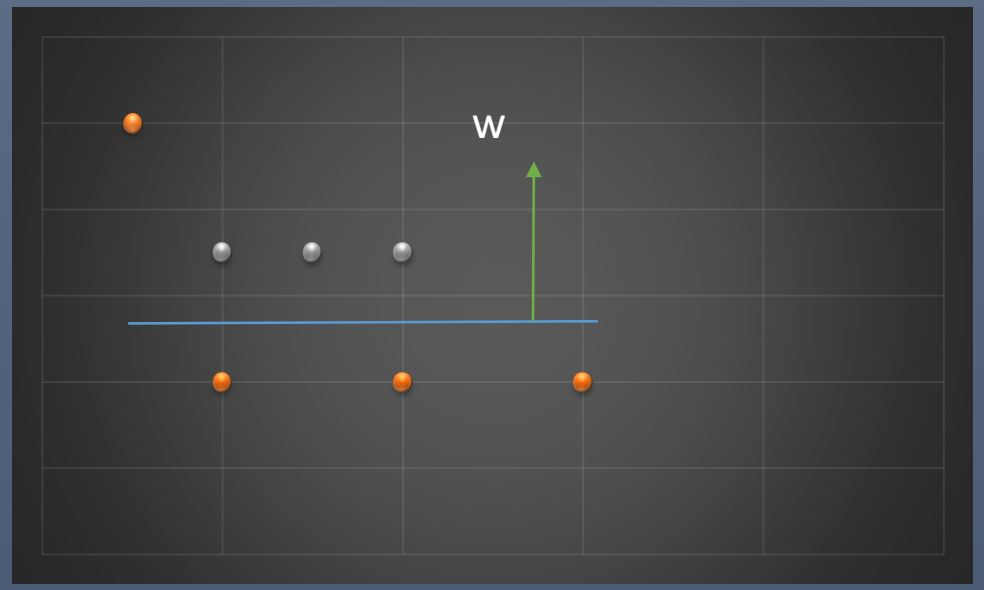
Instead of utilizing a projected point distance of 20 for the outlier point in L2, and +2, +3, -2, -3 predicted point distances for the L1 Line, we will now employ a distance of +1 or -1 based on the positions of the respective points. We will then re-compute the values and make a comparison between the results.



$$\sum_{i=1}^{n} Y_I d_i$$

Original x L1	Final	
1 * 1 = 1	-2	-2
1 * -1 = -1		
1 * -1 = -1		
-1 * 1 = -1	0	
-1 * -1 = 1		
-1 * -1 = 1		
-1 * 1 = -1		

$$\sum_{i=1}^{n} Y_I d_i = 5$$



$$\sum_{i=1}^{n} Y_I d_i$$

Original x L1	Final	
1 * 1 = 1	3	-1
1 * 1 = 1		
1 * 1 = 1		
-1 * 1 = -1	-4	
-1 * 1 = -1		
-1 * 1 = -1		
-1 * 1 = -1		

$$\sum_{i=1}^{n} Y_I d_i = -20$$

It's important to recognize that the process of compressing values, such as transforming

1) +50, +2, and +3 to 1,
2) -2 and -3 to -1,

is accomplished through the utilization of a sigmoid function. This sigmoid function effectively scales down large or infinitely positive/negative values to +1 or -1, respectively. As shown in the below image.

The sigmoid function:

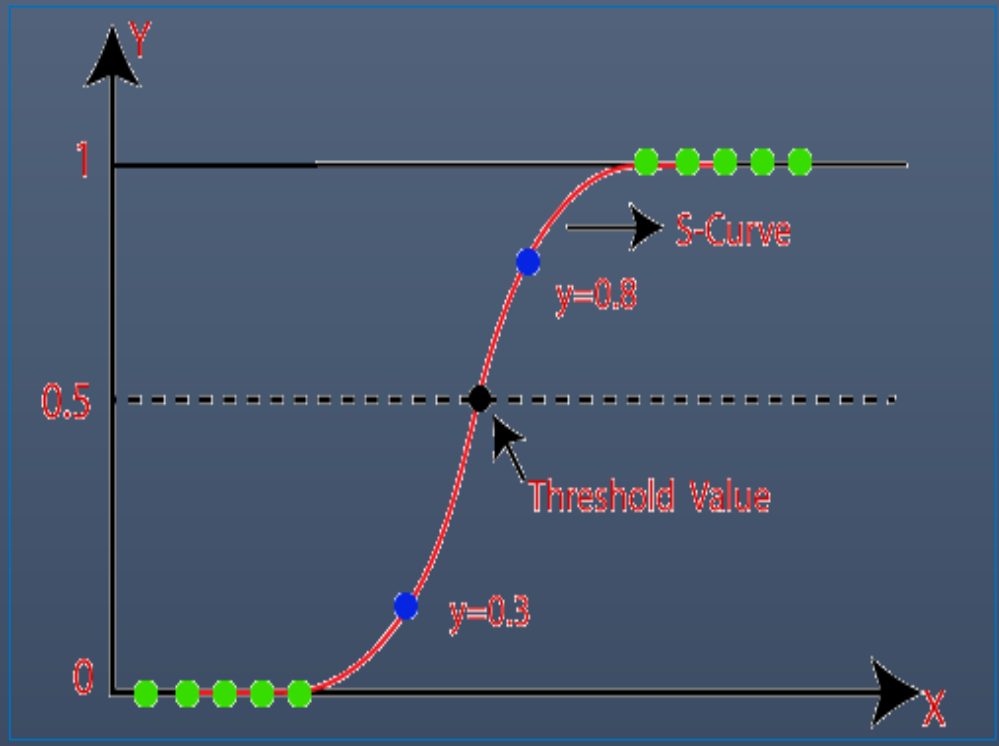
The sigmoid function has the following formula:

$$\sum_{i=1}^{n} log\left(\frac{1}{1-e^{-Z_i}}\right)$$

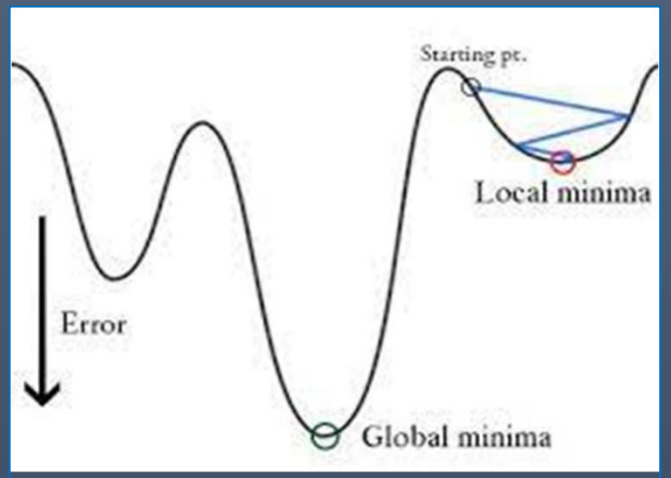
Applying log to the above function and considering

$$Z_i = (W^T X_i)$$

$$\sum_{i=1}^{n} \frac{1}{1-e^{-((W^T X_i)}}$$



The main issue with the above equation is that it generates a non-convex function, leading to numerous local minima (as shown in the below image) and hence restricting the model from reaching the global minima. This, in turn, produces an inaccurate model.



Non-Convex Function

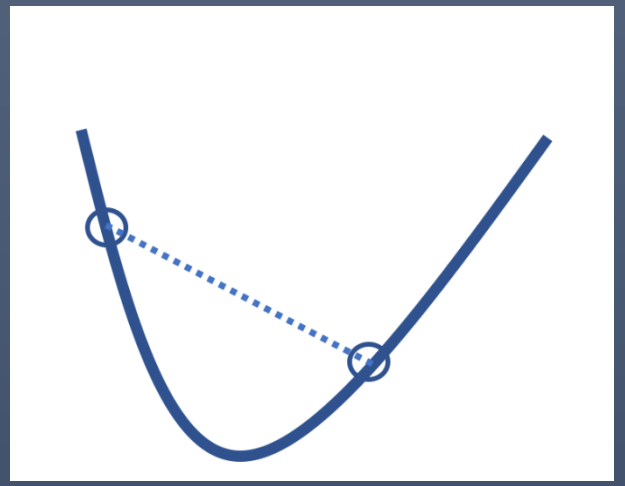
Probabilistic Approach:

Log Loss (Cross entropy) Cost Function:

The main issue with the above equation is that it generates a non-convex function, leading to numerous local minima (as shown in the below image) and hence restricting the model from reaching the global minima. This, in turn, produces an inaccurate model. To tackle this problem, the scientists devised the "**Log Loss Cost Function**" elucidated below.

$$j(\theta) = \frac{1}{m}\left[\sum_{i=1}^{m} -y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

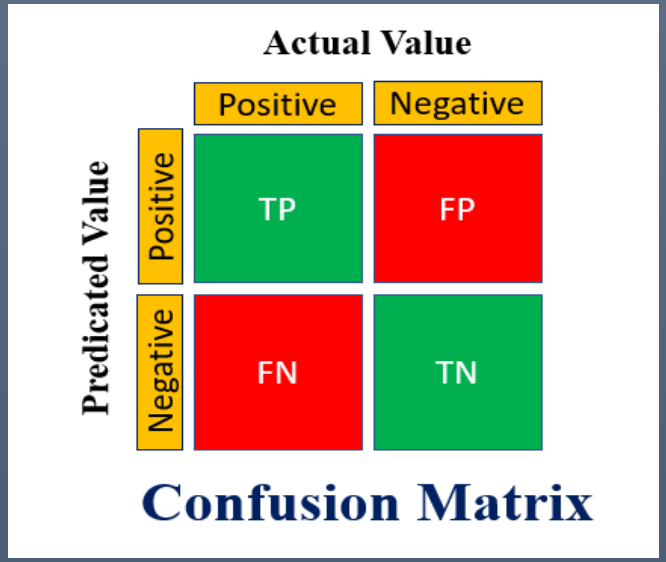
$\hat{y}_i = Predected\ Point$
$y_i = Actual\ Points$
$m = Total\ Number\ of\ points$



The Log Loss Cost Function is a probabilistic approach, it creates a convex function, which has a single global minimum. This property facilitates the model's ability to reach the global minimum easily, enabling the creation of an accurate model.

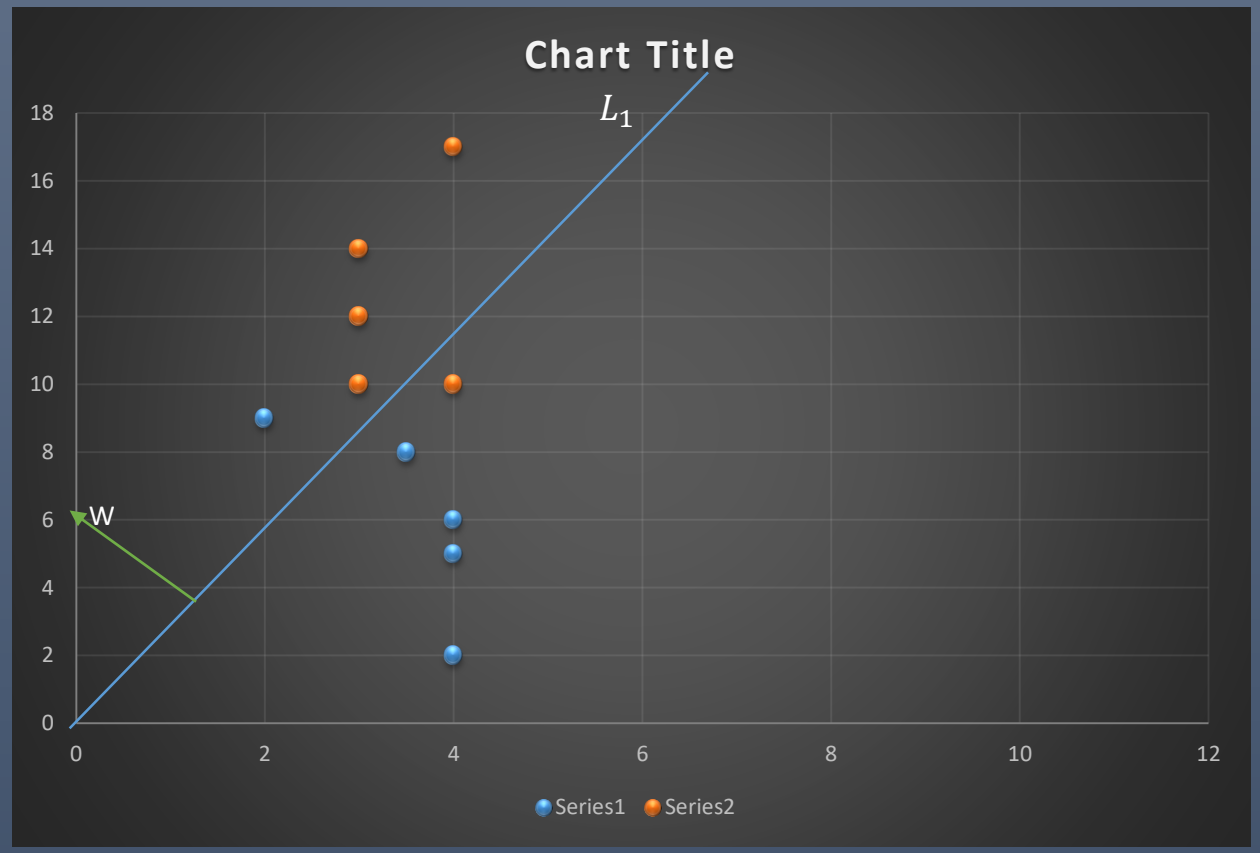
Performance Metrix for classification.

1) Confusion matrix
2) Accuracy
3) Precision
4) Recall
5) F-Beta Square



Confusion matrix :

The confusion matrix can be explained by studying the graph below.



Reading 01	Reading 02	Actual Points(y) (+1 and -1)	Predicted Points(d) (+1 and -1)	+VE or -VE Output	Status For L1
4	17	1	1	1	Correct
3	14	1	1	1	Correct
3	12	1	1	1	Correct
3	10	1	1	1	Correct
4	10	1	-1	-1	Incorrect
2	9	-1	1	-1	Incorrect
3.5	8	-1	-1	1	Correct
6	6	-1	-1	1	Correct
5	5	-1	-1	1	Correct

How to know if the point is correctly classified or not?

To identify if the line has predicted the correct point or not multiply the "actual points" and the "predicted points" and if the result of the same is +ve then we can consider that the particular point is correctly classified and if the result of the same is –ve then consider that the point is incorrectly classified.

$$(Correctly\ Classified\ Points)$$

$$\left\{ \begin{matrix} Y_i(+ve) * d_i(+ve) \\ Y_i(-ve) * d_i(-ve) \end{matrix} \right\} = +ve\ output$$

$$Incorreclty\ Classfied\ Points$$

$$\left\{ \begin{matrix} Y_i(+ve)\ d_i(-ve) \\ Y_i(-ve)\ d_i(+ve) \end{matrix} \right\} = -ve\ Output$$

It's important to recognize that the process of compressing values, such as transforming

1) +50, +2, and +3 to 1,
2) -2 and -3 to -1,

is accomplished through the utilization of a sigmoid function. This sigmoid function effectively scales down large or infinitely positive/negative values to +1 or -1, respectively. As shown in the below image.

The sigmoid function:

The sigmoid function has the following formula:

$$\underset{W = w_1 w_2}{argmax} \sum_{i=1}^{n} \frac{1}{1 - e^{-(Y_i\,(W^T X_i)}}$$

Applying log to the above function and considering $Z_i = Y_i\,(W^T X_i)$

$$\underset{W = w_1 w_2}{argmax} \sum_{i=1}^{n} \log\left(\frac{1}{1 - e^{-Z_i}}\right)$$

$$\underset{W = w_1 w_2}{argmax} \sum_{i=1}^{n} -\log(1 - e^{-Z_i})$$

$$\underset{W = w_1 w_2}{argmin} \sum_{i=1}^{n} \log(1 - e^{-Z_i})$$

The above function is the loss function for logistic regression in the geometric approach. Going further if $Z_i\ is + ve$ then the above equation can rewritten as below

$$\sum_{i=1}^{n} \log\left(\frac{1}{1 - e^{-Z_i}}\right)$$

Here we need to consider that if the output is +ve value then the output will be the minimum

If $Z_i\ is - ve$ then the above equation can rewritten below

$$\sum_{i=1}^{n} \log(1 + e^{Z_i})$$

